

Исследование строковых сортировок

В этой работе я сравнивал обычные алгоритмы сортировки строк и специальные алгоритмы, которые лучше подходят именно для строк.

Обычные сортировки хорошо работают на числах и объектах, которые можно сравнивать за постоянное время. Но для строк ситуация другая: сравнение строк обычно идёт посимвольно. Если строки имеют длинные общие префиксы, то даже одно сравнение может стоить довольно дорого. Поэтому хотелось посмотреть, как на практике ведут себя обычные сортировки и специальные строковые алгоритмы.

Что сравнивалось

- обычный QuickSort;
- обычный MergeSort;
- ternary String QuickSort;
- String MergeSort;
- MSD Radix Sort;
- MSD Radix Sort с переключением на String QuickSort для маленьких подмассивов.

Для каждого алгоритма измерялись:

- время работы;
- количество посимвольных сравнений.

Генерация данных

Для подготовки тестов был сделан класс `StringGenerator`. Он создаёт массивы строк из символов следующего алфавита:

- заглавные латинские буквы;
- строчные латинские буквы;
- цифры;
- специальные символы.

Длина строк выбиралась от 10 до 200 символов. Размеры массивов менялись от 100 до 3000 с шагом 100.

Использовались такие типы массивов:

- случайные;
- обратно отсортированные;
- почти отсортированные;
- массивы с длинными общими префиксами.

Почти отсортированные массивы получались из отсортированного массива с помощью небольшого числа случайных перестановок. Для массивов с общими префиксами специально генерировались строки, у которых начало совпадает, а дальше уже идут случайные символы.

Как проводились измерения

Для измерений был сделан класс `StringSortTester`. Он запускал сортировки на одинаковых входных данных и сохранял результаты в CSV-файл.

Времена замерялись в миллисекундах. Также отдельно считалось число посимвольных сравнений, потому что именно оно хорошо показывает, насколько алгоритм эффективно работает именно на строках.

Результаты сохранялись в файл `data/results.csv`, а потом по ним строились графики.

Ожидания перед экспериментом

От обычных QuickSort и MergeSort можно ожидать, что они будут вести себя более-менее нормально на случайных строках, но на данных с длинными общими префиксами начнут проигрывать. Причина в том, что такие алгоритмы не используют уже найденную информацию о совпадающих началах строк и могут много раз заново сравнивать одни и те же символы.

От String QuickSort ожидалось хорошее поведение именно на строках с общими префиксами, так как он работает посимвольно и делит строки по текущему символу.

От String MergeSort ожидалось улучшение по числу посимвольных сравнений по сравнению с обычным MergeSort.

От MSD Radix Sort ожидалось хорошее время на больших массивах строк, особенно когда строки достаточно длинные.

От версии MSD Radix Sort с переключением на String QuickSort ожидалось, что она будет работать стабильнее, чем чистый MSD Radix Sort, так как на маленьких подмассивах служебные расходы radix-sort уже начинают мешать.

Результаты

Ниже приведены графики времени работы и числа посимвольных сравнений для разных типов данных.

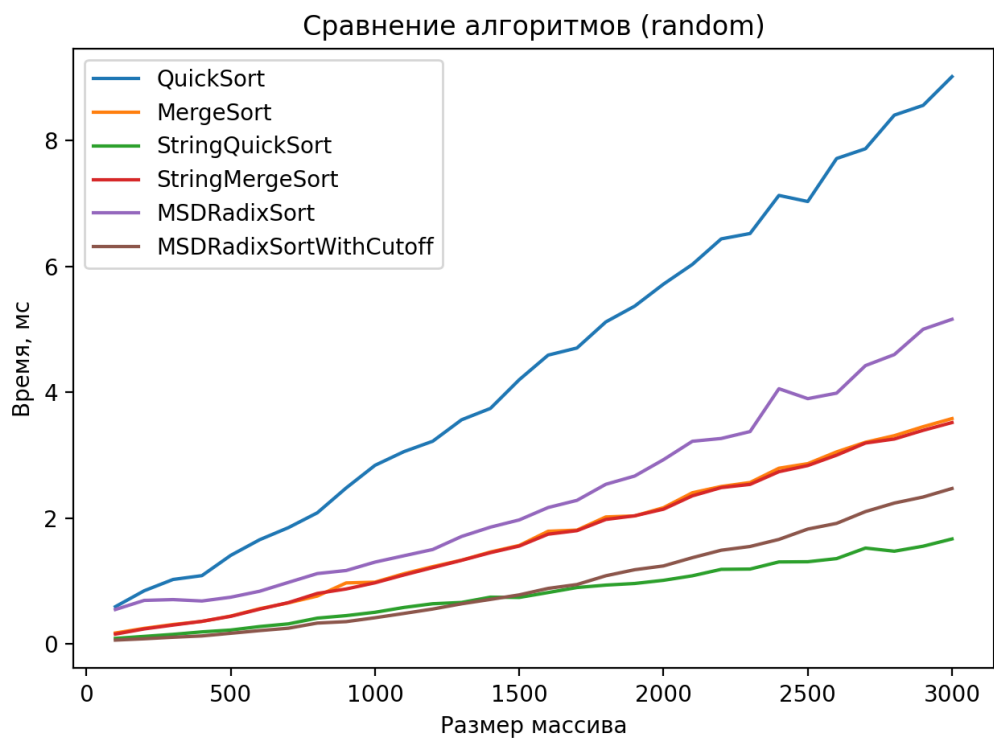


Рис. 1: Время работы на случайных данных

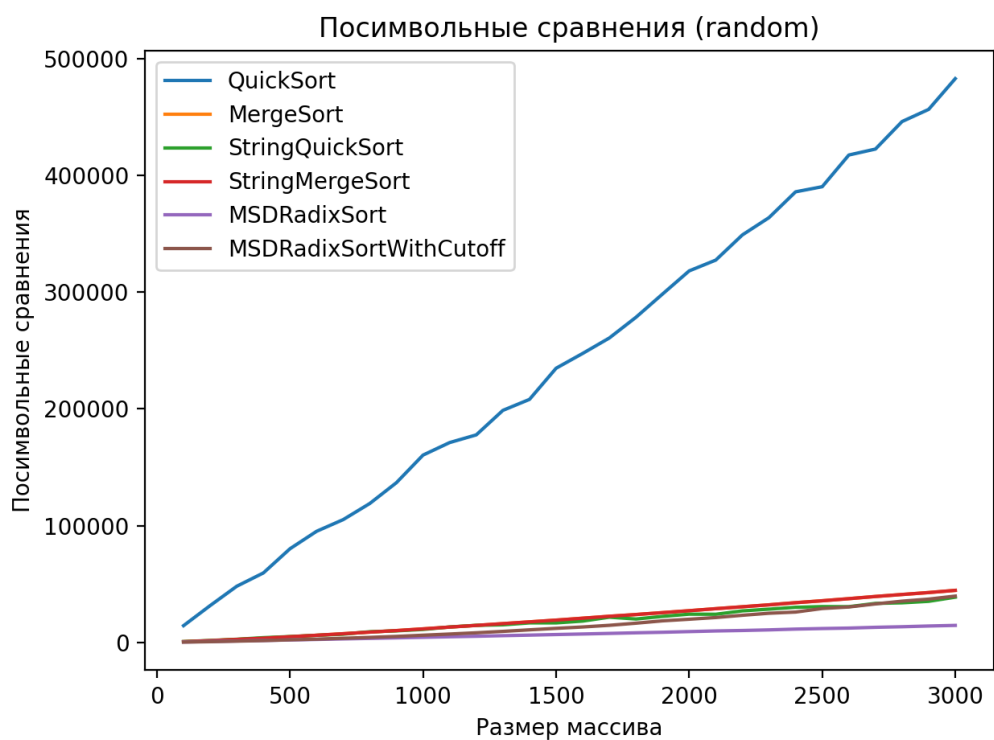


Рис. 2: Посимвольные сравнения на случайных данных

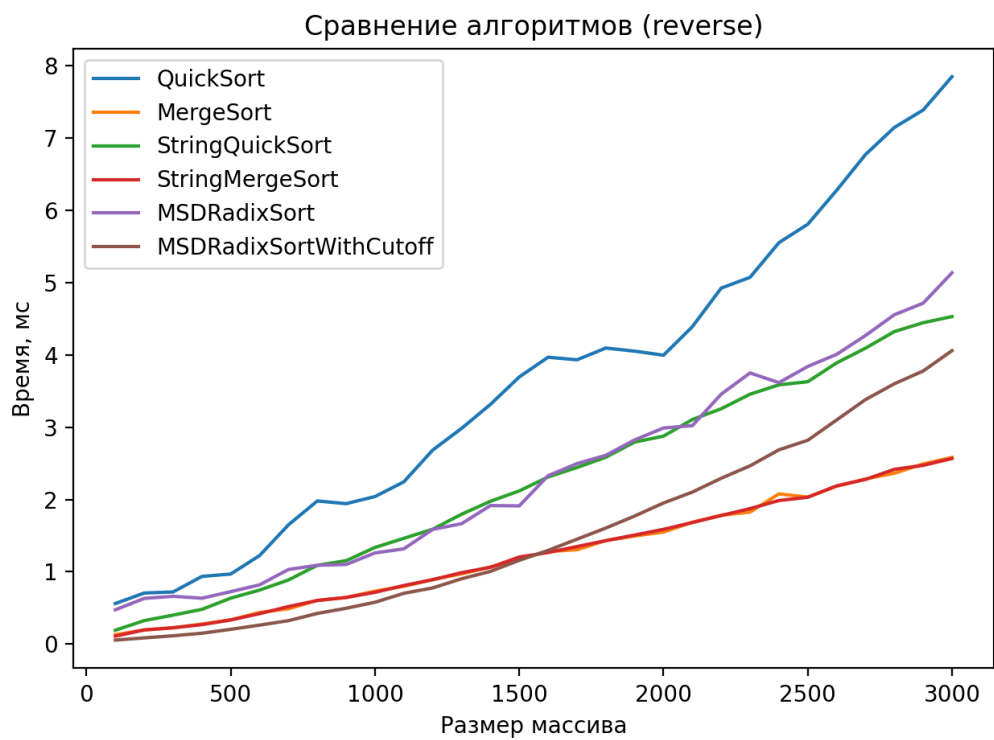


Рис. 3: Время работы на обратно отсортированных данных

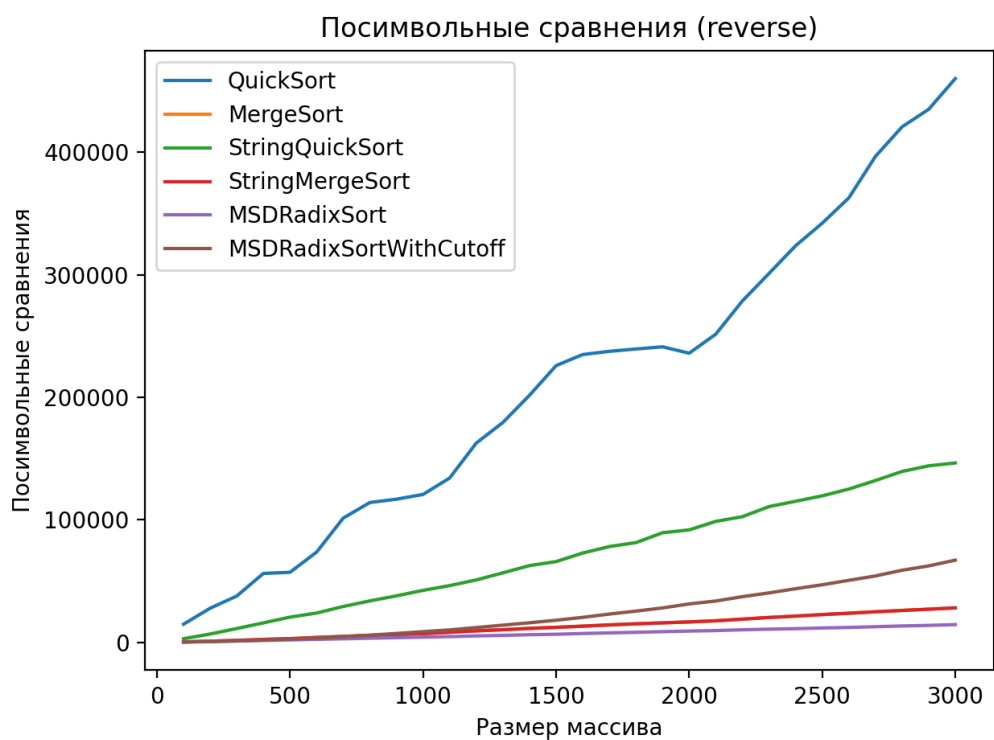


Рис. 4: Посимвольные сравнения на обратно отсортированных данных

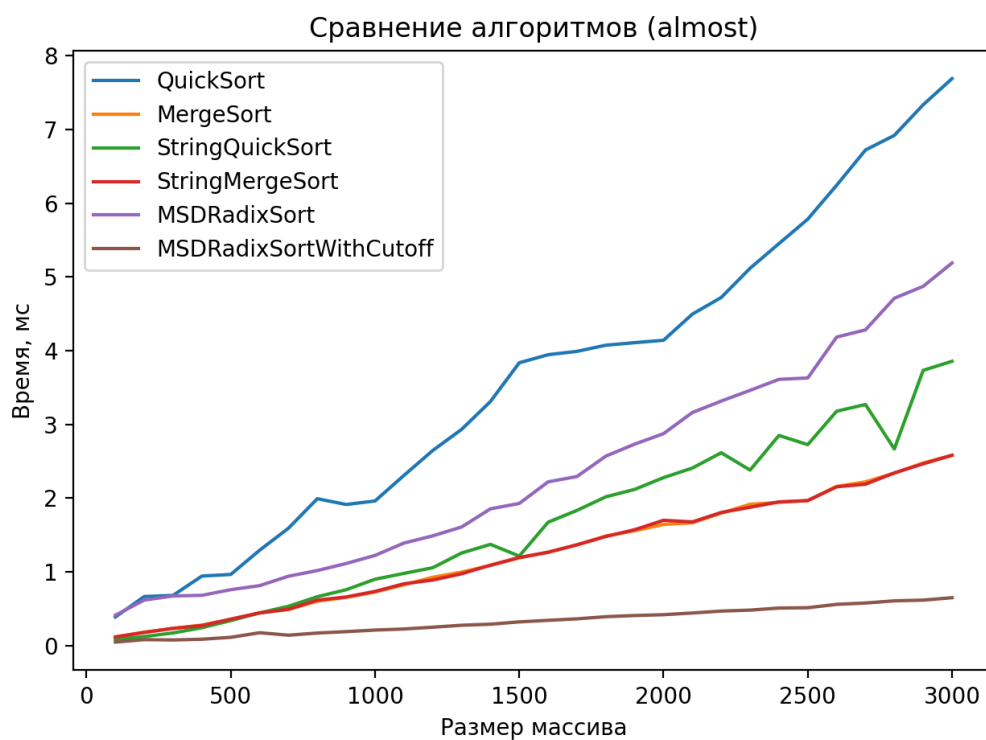


Рис. 5: Время работы на почти отсортированных данных

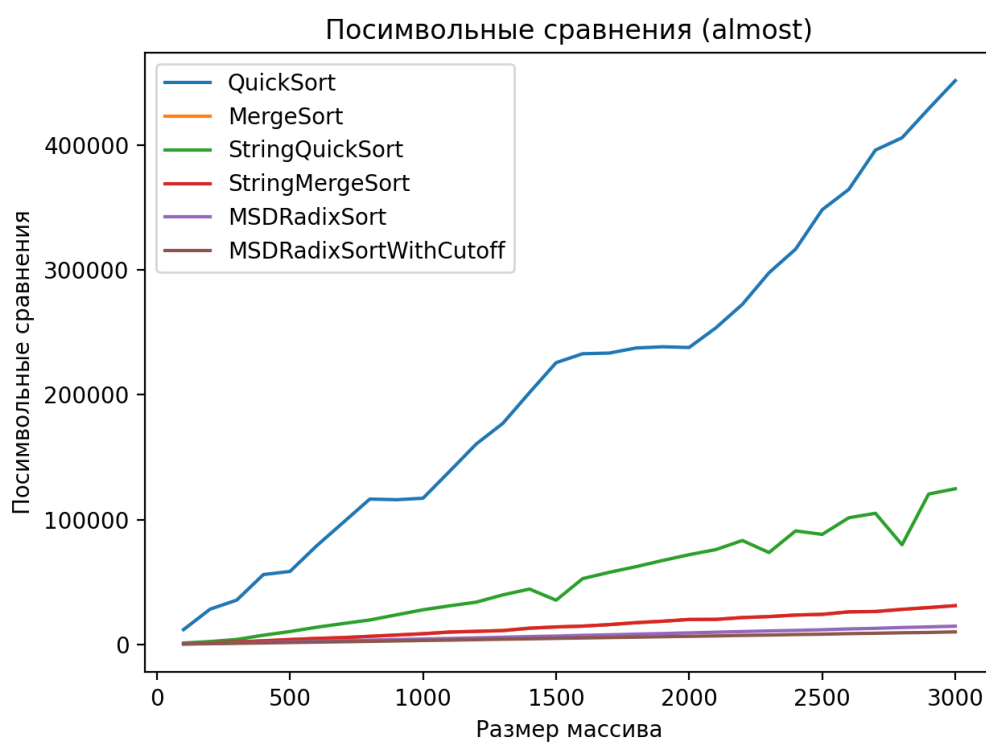


Рис. 6: Посимвольные сравнения на почти отсортированных данных

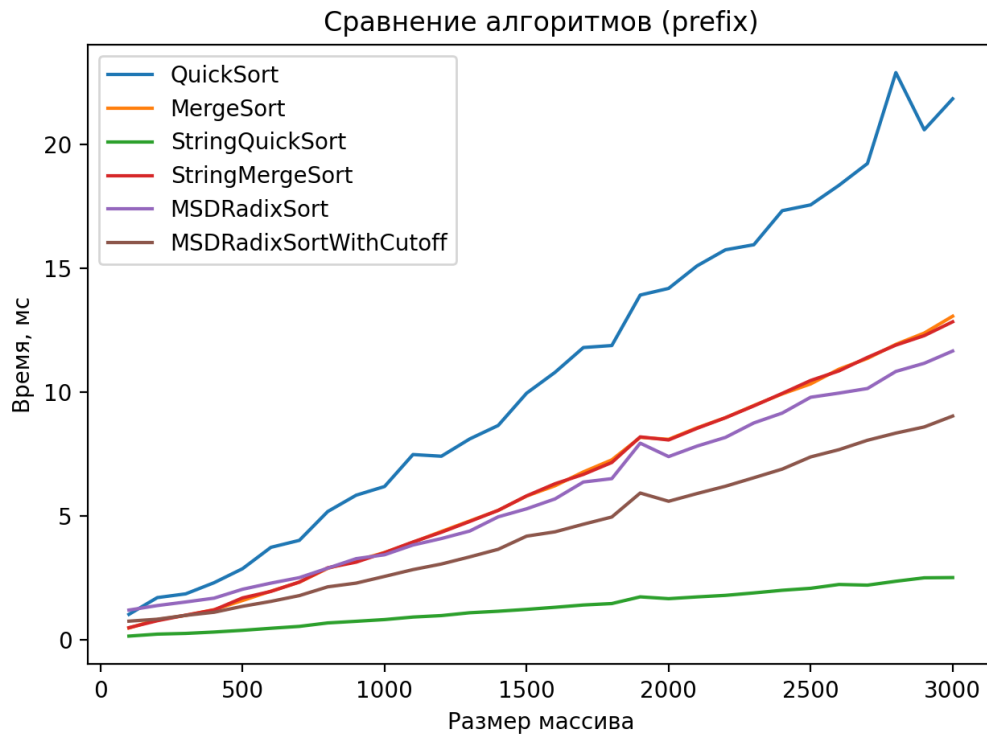


Рис. 7: Время работы на данных с общими префиксами

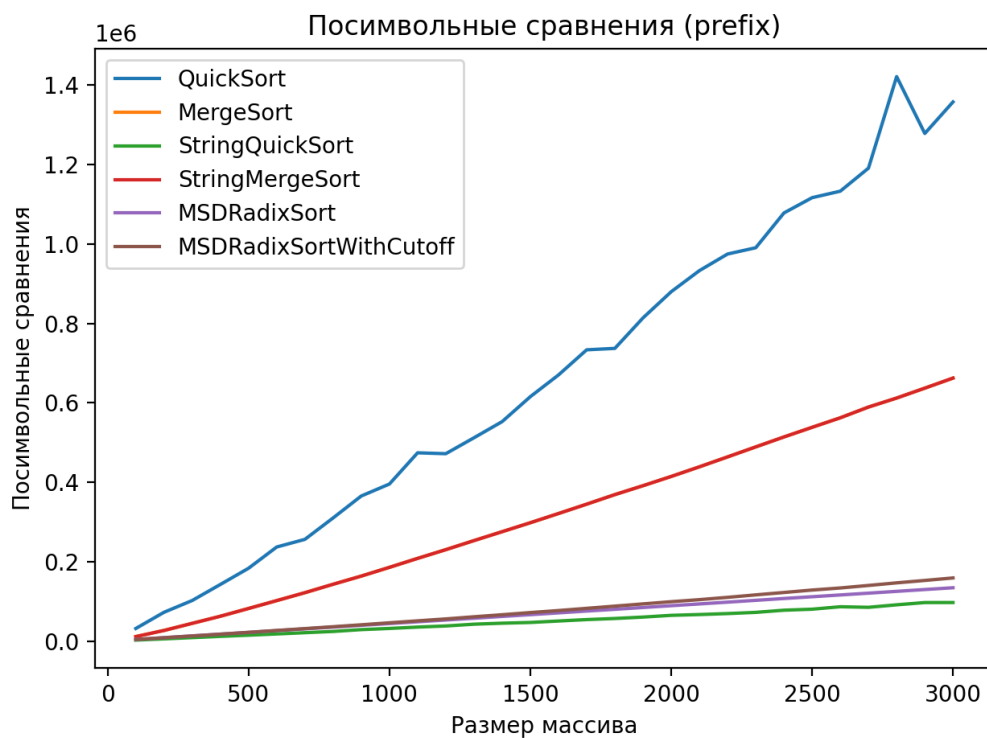


Рис. 8: Посимвольные сравнения на данных с общими префиксами

Что видно по результатам

По случайным данным обычные алгоритмы ведут себя вполне ожидаемо. Разница между стандартными и адаптированными сортировками есть, но она не всегда очень большая.

На обратно отсортированных и почти отсортированных массивах уже лучше видно разницу между подходами. Особенно интересно было посмотреть на число посимвольных сравнений: у строковых алгоритмов оно обычно меньше, чем у обычных QuickSort и MergeSort.

Самая заметная разница проявляется на массивах с длинными общими префиксами. Именно там обычные сортировки начинают делать много лишней работы, потому что одинаковые начала строк заново сравниваются очень много раз. Специальные строковые алгоритмы работают на таких данных лучше.

По результатам видно, что адаптированные алгоритмы действительно оправдывают ожидания. Особенно хорошо это заметно у String QuickSort, String MergeSort и MSD Radix Sort с переключением.

Сравнение с теорией

Полученные результаты в целом соответствуют тому, что ожидалось из теории.

Обычные сортировки на сравнениях имеют хорошую асимптотику по числу сравнений, но для строк этого уже недостаточно, потому что каждое сравнение может быть дорогим.

String MergeSort и String QuickSort лучше используют информацию о префиксах, поэтому уменьшают количество лишних посимвольных сравнений.

MSD Radix Sort вообще работает по символам и не строится на обычных строковых сравнениях, поэтому на длинных строках и больших массивах показывает хорошие результаты.

Версия MSD Radix Sort с переключением на String QuickSort выглядит наиболее практичной, потому что убирает лишние расходы на маленьких подмассивах.

Вывод

По итогам эксперимента можно сделать такой вывод: если сортируются строки, особенно похожие между собой, то обычные QuickSort и MergeSort перестают быть лучшим выбором. На таких данных специальные строковые алгоритмы работают заметно лучше, особенно по числу посимвольных сравнений.

Наиболее интересными по результатам оказались:

- String QuickSort;
- String MergeSort;
- MSD Radix Sort с переключением на String QuickSort.

То есть общий вывод довольно простой: для строк действительно стоит использовать специальные алгоритмы, а не только обычные сортировки сравнениями.

Что приложено

К работе приложены:

- реализации классов `StringGenerator` и `StringSortTester`;
- реализации всех исследуемых алгоритмов;
- CSV-файл с результатами замеров;
- графики;
- ссылка на публичный репозиторий;
- ID посылок A1m, A1q, A1r и A1rq.

ID посылки

- A1m: _____
- A1q: _____
- A1r: _____
- A1rq: _____

Ссылка на публичный репозиторий

`https://github.com/your_login/string-sorting-analysis`